

Construtor PG - Recursos existentes do sistema

Documento detalhado para analista de sistemas

Atualizado em 26/05/2026

Objetivo: oferecer a um analista de sistemas uma visão ampla e operacional dos recursos já existentes, dos fluxos principais, das áreas administrativas, dos controles de segurança e dos pontos que devem ser validados em homologação.

1. Visao geral do produto

O Construtor PG e um motor de sistemas por metadados. A ideia central e permitir que telas, processos, programas administrativos, integracoes e regras operacionais sejam declarados, publicados e executados de forma controlada, com backend decidindo o que o usuario pode acessar e frontend apenas renderizando definicoes autorizadas.

Area	O que existe hoje	Valor para analise
Frontend dinamico	CRUD Engine, Home Engine, Process Engine e Custom Page Engine em HTML simples com Kendo UI/jQuery locais.	Permite validar comportamento de telas sem recompilar a aplicacao.
Backend runtime	Symfony/API Platform/PostgreSQL com screenId, endpointId, autenticacao, sessao, auditoria, jobs e permissoes.	Centraliza seguranca, dados, autorizacao e rastreabilidade.
Construtor	Program Builder para modelar entidades, campos, programas, APIs, regras, historico e publicacao.	Analista consegue transformar requisitos em metadados revisaveis.
Governanca	Solicitacao, grant, bundle de testes, aprovacao, rebase de overlay, retencao e auditoria.	Controla alteracoes em programas padrao e customizacoes por assinante.
Operacao	Provisionamento, instalador, licencas, atualizacoes, integridade, jobs, usuarios, permissoes e parametros.	Cobre ciclo de vida de ambiente e sustentacao.

Principio arquitetural: nenhum JSON de producao deve executar JavaScript livre, usar template livre, expor URL arbitraria ou substituir validacao backend. O frontend monta a experiencia, mas permissao, tenant, dados e transicoes ficam no backend.

2. Stack tecnica e decisoes fechadas

- Frontend em HTML simples, sem build inicial, usando Kendo UI for jQuery local e jQuery local.
- Tema principal atual: `kendo/styles/default-urban.css`.
- Backend em Symfony/API Platform com PostgreSQL, Doctrine, Messenger e comandos CLI.
- Producao inicial usa `screenId` para carregar telas autorizadas e `endpointId/actionId` para acoes.
- A pasta `kendo/` e biblioteca de terceiro e nao deve ser alterada.
- Cultura e mensagens devem permanecer em pt-BR.
- Nao usar `alert`, `confirm` ou `prompt` nativos; confirmacoes devem ser componentes Kendo.
- Nao permitir `eval`, `Function`, template livre ou JavaScript vindo do JSON.

Componente	Tecnologia/arquivo	Observacao
CRUD Engine	src/crud-engine/*	Renderiza grid, filtros, formulario, toolbar, layout, mensagens e chamadas runtime.
Home Engine	src/home-engine/*	Monta appbar, menu lateral, notificacoes, jobs, suporte e abertura de programas.
Process Engine	src/process-engine/*	Executa processos declarativos por parametros com acompanhamento por SSE/polling.
Program Builder	program-builder.html / production/program-builder.html	Modela entidades, programas, APIs, regras, historico e publicacao.
Backend	backend/	Symfony com runtime, admin, autenticacao, migrations, comandos e jobs.

Componente	Tecnologia/arquivo	Observacao
Instaladores	installer/	Executaveis Go por perfil de instalacao.

3. Entradas e navegacao principal

Entrada	Uso	Status atual
login.html / production/login.html	Autenticacao, manter logado, recuperacao de senha, selecao de assinante e escolha de area administrativa.	Demo visual e producao ligada ao backend real.
home.html / production/home.html	Shell principal com menu, appbar, notificacoes, jobs, chat, suporte e abertura de programas.	Home por JSON/screenId com estado local versionado.
index.html	Demo principal de clientes.	Uso demonstrativo.
exemplos.html	Indice central de exemplos e variacoes.	Uso para validacao local.
program-builder.html / production/program-builder.html	Construtor visual de programas.	Interface administrativa principal para autoria.
production/app.html?screenId=...	Entrada generica de producao para CRUD, process e custom.	Carrega somente definicoes autorizadas pelo backend.
production/install.html	Instalacao inicial apos ativacao pelo executavel.	Recusa execucao sem sessao local valida.

4. Login, usuarios, assinantes e sessao

O login ja possui suporte operacional para usuario/senha local, preparacao para LDAP, SSO e OAuth/OIDC, recuperacao de senha, manter logado, selecao de assinante e escolha de area para administrador.

- Autenticacao por `/api/auth/login` com token Bearer vinculado a `runtime\_user\_session`.
- Usuarios ficam em `auth\_user`; provedores em `auth\_provider\_config`.
- Selecao de assinante usa `auth\_subscriber`, `auth\_user\_subscriber` e desafio temporario.
- Administrador pode escolher area principal ou administrativa apos login.
- Manter logado usa `auth\_remember\_token` e endpoint `/api/auth/remember`.
- Recuperacao de senha usa `auth\_password\_reset\_token`.
- Logout, token invalido, sessao revogada, expiracao e force logout limpam contexto local.

Tela administrativa	Finalidade
admin.usuarios	Cadastro de usuarios, status, grupos, permissoes e origem de acesso.
admin.usuario-assinantes	Vinculo usuario-assinante e contexto multi-tenant permitido.
admin.permissoes	Manutencao focada em grupos e permissoes.
admin.sesoes	Consulta de sessoes e acao fechada para derrubar sessao.

5. Home e experiencia operacional

- Appbar com usuario corrente, assinante corrente e recursos contextuais.
- Menu lateral por modulos com busca, favoritos e persistencia do ultimo contexto.

- Abertura de programas por CRUD, process, custom ou iframe controlado.
- Central de notificacoes com filtros de severidade, categoria, acao requerida e nao lidas.
- Jobs no appbar, com abertura para consulta administrativa.
- Chat entre usuarios e atendimento com historico persistido e eventos SSE.
- Suporte por setor, atendente online ou solicitacao persistida.
- Reabertura automatica do ultimo painel contextual quando aplicavel.

6. CRUD Engine e operacao de telas

Recurso	Descricao funcional
Grid Kendo	Paginacao, ordenacao, filtros, acoes de linha, exportacao, agrupamento e selecao.
Filtros	Janela de filtros, filtros salvos, filtros aplicados e edicao de filtros aplicados.
Layout	Persistencia de layout, ordenacao, agrupamento e template mobile por usuario/tenant.
Formulario	Popup com abas, etapas, situacao, logs, impressao, outras acoes e eventos seguros.
Validacao backend	Contrato `validation` + `effects`, destaque de campos e confirmacao por token quando necessario.
Concorrencia	Semaforo/lock, heartbeat, aviso de concorrencia e protecao contra perda de dados.
Mensagens	SSE com fallback por polling para eventos runtime e force logout.
Mobile	Modo colunas e template/card seguro, sem template livre vindo do JSON.

7. Process Engine e jobs assincronos

- Telas `process` coletam parametros declarativos e chamam endpoint fechado.
- Acompanhamento pode ocorrer por SSE ou polling.
- Resultado pode ser mensagem, grid, documento/relatorio ou job iniciado.
- Backend possui Symfony Messenger/Doctrine em PostgreSQL.
- Jobs sao rastreados em `runtime\_async\_job`.
- Exemplos atuais: `cliente.email\_confirmation`, `cliente.whatsapp\_welcome` e `clientes.processamento`.
- Tela `admin.jobs` consulta jobs assincronos.

8. Program Builder

O Program Builder e o principal recurso para construir sistemas. Ele permite modelar modulos, entidades, campos, programas, regras, APIs, historicos, publicacao, governanca e integracoes sem aceitar script livre do usuario.

Capacidade	Detalhe
Modulos estruturais	Cadastro de abreviacao e faixa numerica para validar codigo de programa.
Entidades	`persistence`, `query`, `io` e `api`.
Campos	Tipos, obrigatoriedade, readonly, defaults, FKs, chaves unicas e nomenclatura.
Tabela fisica	Criacao/alteracao controlada, rename, defaults, nullability, precision/scale e rollback.
Importacao PostgreSQL	Lista, inspeciona e importa tabelas existentes como entidade + rascunho CRUD.

Capacidade	Detalhe
Importacao JSON externo	Valida `entityDraft` + `programDraft` antes de carregar para revisao.
Assistente IA	Chat interno com provider mock/openai_compatible, validacao backend e carga de rascunho.
API/Odoo	Entidades API readonly ou CRUD previsivel; Odoo readonly por XML-RPC/JSON-RPC.
Historico	Entidades mestres versionadas e snapshots em `runtime_entity_record_version`.
Codificacao customizada	Campo `custom_code` com padrao declarativo ou metodo restrito no backend.
Regras	Declarativas ou classe/metodo em namespace fechado, com mensagens por literais.

9. Governanca de programas

- Programas possuem ownership e politica de customizacao: `standard`, `customer\_overlay`, `customer\_custom`.
- Programa padrao pode exigir request, grant, bundle de testes e aprovacao final.
- Grant revogado/congelado derruba lock de autoria e bloqueia continuidade.
- Rebase assistido de overlay classifica conflitos como ok, warning ou blocked.
- Conflito leve exige confirmacao explicita; conflito critico bloqueia.
- Retencao da governanca possui preview, aplicacao, historico e comparativo antes/depois.
- Comandos operacionais: `app:governance:monitor`, `app:governance:operations`, `app:governance:cleanup-history`.

Tela	Uso
admin.programa-governanca	Operar requests, grants, bundles, aprovacoes, retencao e rebase.
admin.programa-grants-operacao	Entrada focada para grants.
admin.programa-aprovacoes-operacao	Entrada focada para aprovacoes.
admin.programa-retencao-operacao	Ajuste rapido da retencao.
admin.programa-retencao-historico-operacao	Historico persistido da retencao.
admin.programa-auditoria-operacao	Timeline e sinais operacionais.
admin.programa-overlays-operacao	Overlays, congelamento e rebase.
admin.programa-overlay-versoes-operacao	Historico, comparacao e publicacao de versoes de overlay.

10. Multi-tenant, assinantes e isolamento

- Login pode exigir selecao de assinante quando `subscriber.enabled` estiver ativo.
- Entidades persistentes aceitam `subscriberIsolation.mode=none|subscriber\_column`.
- `subscriber\_column` injeta filtro de assinante em read/get e limita update/delete.
- `none` exige confirmacao explicita de tabela global compartilhada.
- Provisionamento registra deployment mode, ambiente principal e ambiente runtime.
- Modos de deployment: `shared\_program\_shared\_db`, `shared\_program\_dedicated\_db`, `dedicated\_stack`, `onprem\_remote`.
- Tela `admin.assinante-ambientes` mostra matriz operacional e catalogo de isolamento.

11. Integracoes, importacao e exportacao

Item	Detalhe
Tela	`admin.integracoes` com editor visual, TreeView, preview e historico.
Origem	Entidade persistence, API generica, API Odoos readonly e XML declarativo.
Destino	Entidade local, API JSON previsivel, CSV, XML e TXT layout.
TXT	Posicional fixo, delimitado e hierarquico com record/group/totalizer.
XML	Namespaces, atributos, filhos repetitivos, recordPath, xpath e vinculo pai/filho.
Historico	Execucoes, versoes do mapping, agendamentos e exportacao do payload.
Seguranca	Sem transformacao por JavaScript; uso de contratos fechados.

12. Administracao runtime

Tela	Funcao
admin.parametros	Define parametros do sistema, tipos, listas e metadados.
admin.parametro-valores	Valores vigentes globais ou por contexto.
admin.listas-opcoes / admin.opcoes	Listas fechadas e opcoes reutilizaveis.
admin.literais	Literais e traducoes por locale para frontend/backend.
admin.notificacoes	Cadastro de notificacoes runtime.
admin.notificacao-destinatarios	Acompanhamento de entrega e leitura.
admin.integridade	Monitor de assinaturas estruturais e reassinatura controlada.
admin.transacoes / admin.logs-transacoes	Auditoria de operacoes runtime.
admin.jobs	Consulta de jobs assincronos.

13. Integridade, auditoria e rastreabilidade

- Transacoes registram `programVersion`, `builderProgramVersionId`, `builderEntityVersionId`, `screenDefinitionVersion` e `schemaFingerprint`.
- Tambem registram ambiente, identidade do banco, tipo de customizacao, grant, request, approval e bundle de teste quando aplicavel.
- Assinatura estrutural em `system\_record\_integrity` cobre programas, entidades, campos, endpoints, overlays, parametros, opcoes, integracoes e outros registros sensiveis.
- Comandos: `app:integrity:check`, `app:integrity:monitor`, `app:integrity:resign`.
- Reassinatura controlada registra motivo, usuario, horario, hash anterior e status antes/depois.

14. Instalacao, licencas e provisionamento

Recurso	Detalhe
Executaveis Go	Quatro binarios: builder/subscriber para Linux e Windows; perfil compilado.

Recurso	Detalhe
Precheck	Valida dependencias por modo; ERRO bloqueia e AVISO permite continuar registrado.
Ativacao central	Codigo do assinante, e-mail de confirmacao, sessao curta e manifesto autorizado.
Licencas	`admin.instalacao-licencas` controla e-mail, perfil, modo, validade, status e limite.
Tokens internos	`admin.instalacao-tokens` controla tokens para provisionamento SaaS sem e-mail manual.
Operacoes da central	`admin.central-operacoes` consolida painel operacional, auditoria, revogacao, tentativas/bloqueio, chaves, artefatos, saude dos assinantes e notificacoes derivadas.
Bloqueio de tentativas	Codigos de e-mail invalidos sao bloqueados por requisicao conforme `APP_INSTALLER_ACTIVATION_MAX_ATTEMPTS` e `APP_INSTALLER_ACTIVATION_BLOCK_MINUTES`.
Pagina web	`production/install.html` mostra ativacao e executa etapas finais.
Docker Linux	`app` com Nginx/PHP-FPM/Supervisor e `database` PostgreSQL 16.
Worker	No Docker fica inativo por padrao; ativar com `APP_WORKER_ENABLED=1` apos instalacao.
Reinstalacao	Exige nova ativacao, senha do instalador e confirmacao explicita.

A pagina de instalacao nao pede mais token de liberacao. O bloqueio e feito pela sessao local criada pelo executavel. Sem sessao valida, `/api/install/run` recusa a instalacao.

15. Atualizacoes do sistema

- Tela `admin.atualizacoes` le manifesto, avalia dependencias e aplica releases por job.
- Tela `admin.atualizacoes-assinantes` consulta historico por assinante.
- Releases aceitam `requiresVersionMin`, `requiresAppliedUpdates`, `replaces`, `category`, `autoApply`, `breakingLevel` e `steps`.
- Politica da release declara backup, manutencao, auto apply, anuencia, bloqueio de proximas e internet obrigatoria.
- Manifesto passa por validacao de coerencia: dependencias inexistentes, replaces invalidos, ciclos e auto-referencia.
- SaaS pode usar rollout por janela, batches/canario e orquestrador externo assinado.
- On-premise usa runner `update-onprem.sh|ps1` e politicas criticas de warn/block, auto/prompt/download_only.
- Programas `standard`, `customer\_overlay` e `customer\_custom` respeitam regras diferentes de atualizacao.

16. Seguranca funcional

Regra	Impacto
screenId em producao	Frontend pede uma tela conhecida; backend devolve apenas definicao autorizada.
endpointId/actionId	Acoes passam por identificadores fechados, evitando URL livre no JSON.
Sem JS livre	JSON nao pode injetar `eval`, `Function`, template livre ou script.
Permissao backend	Permissao visual nao substitui validacao de usuario, tenant, registro e transicao.
Auth required	`AUTH_REQUIRED=1` recusa runtime sem token valido.
Segredos	Tokens e chaves devem ficar em parametros/variaveis mascaradas.
Instalacao	Executavel + central + sessao curta reduzem risco de liberacao manual indevida.

17. Roteiro sugerido para analise funcional

Ordem	Trilha	Resultado esperado
1	Instalacao/licenca	Ambiente ativado, precheck ok, pagina install liberada e sistema instalado.
2	Login/sessao	Usuario entra, seleciona assinante e area correta.
3	Home	Menu, appbar, notificacoes, jobs e contexto funcionam.
4	CRUD base	Grid, filtro, formulario, validacoes e acoes passam.
5	Processos/jobs	Processamento por parametros e job assincrono acompanham corretamente.
6	Program Builder	Modulo, entidade, programa, preview e publicacao funcionam.
7	Governanca	Request, grant, teste, aprovacao, publish e rebase cobertos.
8	Integracoes	Mapping, preview, execucao, historico e agendamento validados.
9	Integridade	Monitor sem invalidez ou com fluxo de reassinatura controlado.
10	Atualizacoes	Manifesto, precheck, simulacao, apply, rollback e historico por assinante.

18. Evidencias que o analista deve coletar

- Ambiente usado: demo local, producao local, backend real, on-premise ou SaaS.
- Usuario, perfil, grupos e assinante usados em cada trilha.
- `screenId` e programa aberto.
- Passos executados e resultado esperado x obtido.
- Screenshots dos fluxos principais.
- Erros separados por tipo: funcional, permissao, dado, ambiente, documentacao ou seguranca.
- Quando houver instalacao: licenca usada, modo, precheck, ativacao, data/hora e resultado.
- Quando houver publicacao: versao, request, grant, bundle de teste e aprovacao.
- Quando houver update: release, politica, simulacao, aplicacao, rollback e impacto em overlays.

19. Mapa rapido de programas e telas

Grupo	Telas principais
Acesso	production/login.html, admin.usuarios, admin.usuario-assinantes, admin.permissoes, admin.sesoes
Navegacao	production/home.html, production/app.html?screenId=...
Operacao diaria	cadastros.clientes, admin.jobs, telas CRUD publicadas pelo builder
Construcao	production/program-builder.html
Governanca	admin.programa-governanca e entradas focadas de grants, aprovacoes, retencao, auditoria e overlays
Administracao	admin.parametros, admin.parametro-valores, admin.literais, admin.notificacoes, admin.integridade
Integracoes	admin.integracoes

Grupo	Telas principais
Provisionamento	admin.assinante-ambientes, admin.instalacao-licencas, admin.instalacao-tokens, admin.central-operacoes, production/install.html
Atualizacoes	admin.atualizacoes, admin.atualizacoes-assinantes

20. Pontos de atencao para proximas homologacoes

- Confirmar se migrations e `app:seed-runtime-metadata` foram executados no ambiente alvo.
- Confirmar SMTP real antes de validar ativacao por e-mail e recuperacao de senha.
- Confirmar `AUTH\_REQUIRED=1` quando validar producao real.
- Confirmar worker ativo quando a trilha envolver jobs, provisionamento ou atualizacao.
- Confirmar se o ambiente e central SaaS quando validar telas administrativas restritas ao central.
- Confirmar se a porta Docker publicada nao conflita com outro servico local.
- Confirmar se licencas de instalacao usam limites coerentes para o contrato do assinante.
- Confirmar se tabelas globais foram explicitamente justificadas no builder.
- Confirmar se programa padrao nao foi alterado sem governanca quando a politica exigir gate.

21. Checklist detalhado de validacao por area

Area	Validacoes minimas
Instalacao	Licenca ativa, perfil correto, modo autorizado, e-mail recebido, precheck sem ERRO, sessao local gravada, install finalizado.
Central operacional	Painel sem alerta critico, chaves fortes, artefatos configurados, token SaaS valido, auditoria e saude dos assinantes revisadas.
Login	Senha valida, senha invalida, manter logado, recuperar senha, limpar sessao local, expiracao e logout.
Assinante	Selecao apos login, troca pela Home quando habilitada, permissao por assinante e fallback para principal.
Home	Menu, favoritos, busca, ultimo programa, notificacoes, jobs, chat, suporte e persistencia de contexto.
CRUD	Read, get, create, update, delete, validacoes, concorrancia, filtros, layout, exportacao e mobile.
Processo	Parametros obrigatorios, validacao, inicio, acompanhamento, cancelamento/erro, retorno e documento.
Builder	Modulo, entidade, campo, tabela, regra, preview, diagnostico, publish e abertura por screenId.
Governanca	Request, grant, lock, bundle de testes, aprovacao, publish, auditoria e retencao.
Overlay	Criacao por assinante, rebase, conflito leve, conflito bloqueante, congelamento e publish.
Integracao	Cadastro de mapping, preview, execucao, historico, versao, agenda e exportacao.
Integridade	Monitor, item valido, item invalido, reassinatura, log e comando CLI.
Atualizacao	Check, simulacao, precheck, anuencia, aplicacao, rollback, impacto em programa e timeline.

22. Matriz de recursos do Program Builder

Frete	Recursos existentes	O que o analista deve observar
Modulo	Abreviacao, faixa numerica, agrupamento estrutural.	Codigo de programa precisa estar dentro da faixa e seguir padrao.
Entidade persistence	Tabela fisica, campos, PK, FKs, defaults, unique, readonly, situacao.	Sem metadados completos o runtime nao deve inferir tela automaticamente.
Entidade api	API cadastrada, operacoes de lista/detalhe/escrita, jsonPath, Odoos readonly.	Validar se contrato externo e previsivel e se nao ha transformacao livre.
Programa CRUD	Grid, formulario, filtros, permissoes, endpointId, preview.	Confirmar se a definicao publicada abre em producao por screenId.
Programa custom	Entrada relativa iframe/htmlUrl autorizada.	Nao aceitar URL externa livre em producao.
Programa process	Parametros, endpoint process/status, retorno fechado.	Confirmar job e acompanhamento por SSE/polling.
Historico	Snapshot de mestre e referencia historica em transacional.	Validar que registro antigo continua mostrando dado da epoca.
Codificacao	Pattern declarativo, sequencia, assistente seguro, metodo estatico restrito.	Valor final deve ser gerado no backend.
Regras	requiredWhen, class_method, ordem, fase, continueOnError.	Mensagens devem preferir messageKey/messageParams.
Publicacao	Draft, published, archived, duplicacao, rollback e gate de ambiente.	Publicar programa padrao pode exigir governanca.

23. Matriz de instalacao e operacao

Cenario	Fluxo	Ponto critico
Linux Docker on-premise	Executavel Linux, precheck Docker, ativacao, compose, pagina install.	Docker, Compose, portas, registry, disco, DNS e relógio.
Linux nativo on-premise	Executavel Linux, precheck PHP/Composer/PostgreSQL, pacote assinado, servico local.	PHP 8.4, extensoes, psql/pg_dump/pg_restore, systemd e permissoes.
Windows teste	Executavel Windows, native mode, servidor local simples.	Nao e producao; nao oferecer modo Docker.
Docker SaaS	Orquestrador usa token interno, manifesto autorizado e stack do assinante.	Sem e-mail manual; token e assinatura precisam estar corretos.
Reinstalacao	Nova ativacao, senha do instalador e confirmacao explicita.	Exigir backup ou justificativa operacional.
Atualizacao on-premise	Runner local consulta manifesto, baixa pacote, aplica steps e recria containers se preciso.	Politica critica warn/block e modo auto/prompt/download_only.
Rollout SaaS	Central despacha plano para orquestrador externo por HTTP assinado.	Batches, janela, canario, bloqueio temporario e auditoria.

24. Matriz de seguranca e controles

Controle	Como funciona	Risco mitigado
screenId	Backend resolve tela conhecida e autorizada.	Carga de JSON livre ou tela nao autorizada.
endpointId	Acoes passam por identificadores publicados.	Chamada direta a URL livre pelo JSON.

Controle	Como funciona	Risco mitigado
Auth Bearer	Sessao runtime vinculada ao token.	Uso anonimo indevido quando AUTH_REQUIRED=1.
Permissao	Backend valida tela, endpoint, tenant, usuario e registro.	Permissao apenas visual no frontend.
Tenant	Assinante selecionado filtra ou limita operacoes.	Vazamento entre assinantes.
Integridade	Assinatura estrutural detecta alteracao fora do fluxo.	Mudanca manual de metadados sensiveis.
Governanca	Request, grant, teste e aprovacao para programa padrao.	Alteracao padrao sem controle.
Instalador	Executavel, licenca, e-mail, sessao curta e prova assinada.	Instalacao liberada por alteracao simples de env.
Update	Manifesto assinado, pacote assinado, politica e precheck.	Aplicacao de release incoerente ou nao autorizada.

25. Perguntas que o analista deve responder

- O sistema esta sendo validado em demo, producao local, SaaS ou on-premise?
- O login usado representa corretamente usuario comum, administrador e assinante?
- A Home mostra os programas esperados para o perfil testado?
- As telas administrativas aparecem apenas para usuarios autorizados?
- As entidades persistentes por assinante possuem filtro correto ou foram assumidas como globais com justificativa?
- O Program Builder consegue transformar um requisito novo em entidade, programa, preview e tela publicada?
- O fluxo de governanca impede alteracao indevida em programa padrao?
- As integracoes usam contratos fechados e historico de execucao suficiente?
- As atualizacoes respeitam cadeia, anuencia, backup, manutencao e impacto em customizacoes?
- A instalacao exige licenca, ativacao, precheck e sessao local antes da tela web?
- As evidencias coletadas permitem reproduzir cada erro encontrado?

26. Apendice: comandos e arquivos de referencia

Finalidade	Comando/arquivo
Bootstrap	php backend/bin/console app:install:bootstrap
Criar assinante	php backend/bin/console app:subscriber:create
Publicar defaults	php backend/bin/console app:runtime:publish-defaults
Seed runtime	php backend/bin/console app:seed-runtime-metadata --no-interaction
Worker	php backend/bin/console messenger:consume async -vv
Integridade	php backend/bin/console app:integrity:monitor --fail-on-invalid
Governanca	php backend/bin/console app:governance:operations
Atualizacao	php backend/bin/console app:update:check / app:update:apply
Instalador	installer/build.ps1 ou installer/build.sh
Docker	docker compose build / docker compose up -d
Manual instalacao	docs/manual-instalacao.md
Roteiro analista	docs/roteiro-validacao-funcional-analista.md